

OSSS-3

Question:

1. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Solution:

```
ubuntu@ubuntu:~$ pwd
/home/ubuntu
ubuntu@ubuntu:~$ date
Fri Sep 18 08:05:47 UTC 2026
ubuntu@ubuntu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos snap
ubuntu@ubuntu:~$ whoami
ubuntu
ubuntu@ubuntu:~$ history
 1 history
 2 clear
 3 pwd
 4 date
 5 ls
 6 whoami
 7 history
ubuntu@ubuntu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos snap
ubuntu@ubuntu:~$ !!
ls
Desktop Documents Downloads Music Pictures Public Templates Videos snap
ubuntu@ubuntu:~$ !5
ls
Desktop Documents Downloads Music Pictures Public Templates Videos snap
```

```
ubuntu@ubuntu:~$ history > history.txt
```

```
ubuntu@ubuntu:~$ cat history.txt
```

```
1 history
```

```
2 clear
```

```
3 pwd
```

```
4 date
```

```
5 ls
```

```
6 whoami
```

```
7 history
```

```
8 ls
```

```
9 history > history.txt
```

```
ubuntu@ubuntu:~$ nano escape.c
```

```
ubuntu@ubuntu:~$ gcc escape.c -o escape
```

```
ubuntu@ubuntu:~$ ./escape
```

```
New Line Example
```

```
Tab      Example
```

```
Backslash: \
```

```
Double Quote: "Hello"
```

```
GNU nano 8.7.1
```

```
escape.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("New Line Example\n");
```

```
    printf("Tab\tExample\n");
```

```
    printf("Backslash: \\n");
```

```
    printf("Double Quote: \"Hello\"\n");
```

```
    return 0;
```

```
}
```

```
ubuntu@ubuntu:~$ nano dynamic_memory.c
ubuntu@ubuntu:~$ gcc dynamic_memory.c -o dynamic_memory
ubuntu@ubuntu:~$ ./dynamic_memory
Enter size: 9
Array Elements:
1 2 3 4 5 6 7 8 9
Memory Resized Successfully
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;

    printf("Enter size: ");
    scanf("%d",&n);

    int *arr = (int *)malloc(n * sizeof(int));

    if(arr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for(int i=0;i<n;i++)
        arr[i] = i+1;

    printf("Array Elements:\n");

    for(int i=0;i<n;i++)
        printf("%d ",arr[i]);

    printf("\n");

    arr = realloc(arr,(n+5)*sizeof(int));

    printf("Memory Resized Successfully\n");

    free(arr);

    return 0;
```